

Cardano SeedSigner: Air-Gapped Hardware Wallet Application Specification

Document Version:	1.1
Author:	Angel Castillo
Date Created:	2026/01/07
Last Updated:	2026/07/20

Revision History

Version	Date	Changes
1.0	2026/01/07	Initial specification.
1.1	2026/07/20	Protocol chapter updated to the protocol as implemented and shipped: exact CDDL for all request/response payloads (per-signer master fingerprints, change outputs, extra signers, collateral return path; the earlier conceptual “resolved inputs” field was dropped), account export request/response schemas, CIP-8 COSE wire format (Ledger/Keystone parity), payload validation rules, ownership verification and review badges, curated verified-asset display, corrected seed fingerprint definition, updated menu and settings tables (15-word seeds, multi-account and multi-credential gates, Compact SeedQR, warning toggles).

Table of Contents

Overview

- Project Overview
- Relationship to Bitcoin SeedSigner
- Trust Boundaries
- User Consent and Authorization

Hardware Architecture

- Supported Hardware
- Hardware Compatibility Matrix
- Hardware Interaction Model
- Optional MicroSD Usage
- Power and Lifecycle

Application Architecture

- Application Structure
- State Management

User Interface & Menus

- User Interaction Principles
- Main Menu Overview
- Scan Flow
- Seed Management
- Tools Menu
- Transaction Signing Flow
- Transaction Review Sections
- Ownership Verification & Review Badges
- Verified Assets
- Message Signing Flow
- Settings & Device Configuration
- Error and Warning Screens

Cardano Key & Signing Model

- Supported Key Types
- Derivation & Identification
- Fingerprints / Key Identification

QR Communication & UR Encoding Model

- Communication Pattern
- UR URI Format

Encoding Building Blocks

UR Types Used by Cardano SeedSigner

Payload Definitions (CDDL)

CIP-8 COSE Wire Format

Validation Rules

Fragmentation & Animated QR

Compatibility & Extensibility

User & System Interaction Diagrams

Seed Import

Extended Public Key (xpub) Export

Settings Import/Export

Transaction Signing

CIP-8 Message Signing

Overview

Project Overview

Cardano SeedSigner is a DIY, fully air-gapped hardware wallet application designed to enable secure key management, transaction signing, and message signing for the Cardano blockchain using animated QR codes.

The project extends the existing SeedSigner execution model to Cardano, preserving its stateless operation, offline security guarantees, and user-mediated authorization flow while adapting it to Cardano-specific transaction semantics, signing requirements, and data structures.

This document defines the technical architecture, user experience flows, and QR-based communication protocols required to support Cardano while maintaining SeedSigner's core security principles. Version 1.1 of this document describes the protocol exactly as implemented by the shipped firmware.

Relationship to Bitcoin SeedSigner

Cardano SeedSigner is based on the Bitcoin SeedSigner project, which serves as the primary architectural reference for this work. Bitcoin SeedSigner demonstrates a fully offline signing device that relies exclusively on animated QR codes for data transfer and enforces a strict separation between offline signing and online transaction construction.

This project reuses SeedSigner's proven hardware abstractions, stateless execution model, and minimal trusted computing base. Where necessary, the design adapts these concepts to Cardano-specific requirements, such as different key roles, transaction structures, and signing contexts, without introducing new cryptographic primitives or transport mechanisms.

See <https://seedsigner.com/>

Trust Boundaries

The online wallet software is treated as untrusted. The device does not assume the correctness of host-provided metadata such as UTxO ownership, balances, or transaction intent beyond what can be cryptographically verified or clearly presented to the user.

Host claims that can be verified are verified: outputs claimed as change and the collateral return output are re-derived from the loaded seed and only labelled as the wallet's own when the derived address matches the transaction body exactly; credentials appearing in certificates, withdrawals, required signers, and votes are compared against key hashes derived from the loaded seed. A claim that fails verification degrades safely: the item is presented as foreign/unknown, never hidden.

User Consent and Authorization

The user experience is designed around explicit user consent rather than inferred intent. Every signing operation requires direct user approval based on information shown on the device itself. The device does not autonomously infer ownership or intent and does not rely on unverifiable claims from the host.

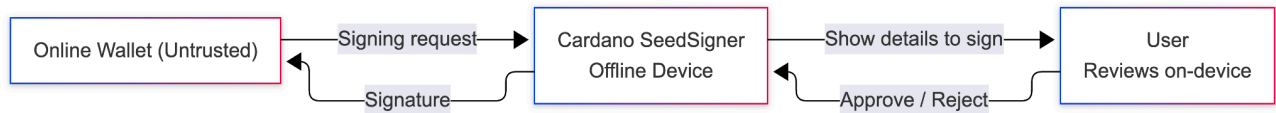


Figure 1 Transaction/Message sign flow.

This approach ensures that authorization decisions remain user-driven, inspectable, and resistant to manipulation by compromised or malicious online software.

The user is also able to export public data to the outside world, example:



Figure 2 Exporting the account public key.

Hardware Architecture

Supported Hardware

Cardano SeedSigner is designed to run on commodity, widely available hardware with no proprietary components. The reference platform is the Raspberry Pi family, selected for its open ecosystem, strong community support, and compatibility with the existing SeedSigner hardware model.

Supported configurations include:

- Raspberry Pi Zero class devices and newer
- SPI-connected color LCD displays
- Raspberry Pi Camera Module (CSI interface)
- Physical GPIO buttons for all user input
- Optional MicroSD card for non-sensitive settings storage

The hardware design deliberately excludes networking interfaces, secure elements, and persistent secret storage. All cryptographic material exists only in volatile memory for the duration of a powered session.

Hardware Compatibility Matrix

The following table summarizes the Raspberry Pi platforms and display combinations that are supported:

Raspberry Pi Model	GPIO Header	Display Support	Camera Support
Pi Zero / Zero W	40-pin	ST7789	v1.3 / v2
Pi Zero 2 W	40-pin	ST7789	v1.3 / v2
Pi 3 (all variants)	40-pin	ST7789 / ILI9341	v1.3 / v2
Pi 4 (all variants)	40-pin	ST7789 / ILI9341	v1.3 / v2

Hardware Interaction Model

The device interacts with the user and the external environment exclusively through a small, well-defined set of hardware interfaces. These interfaces are used only for local input and output and never for network communication.

Button Input

All user input is performed via physical GPIO buttons, arranged as a 5-way directional pad with additional action buttons.

Button inputs are configured as active-low GPIO signals using internal pull-up resistors. All critical actions, including transaction approval, key management, and navigation, require explicit physical interaction. No remote or automated input paths exist.

Representative GPIO pin assignments:

Pin	Function
31	Up
35	Down
29	Left
37	Right
33	Select
40	Back
38	Power

Display Output

The display is a SPI-connected color LCD and serves as a security-critical output surface. All information required for user verification, addresses, transaction details, stake actions, warnings, and QR codes, is rendered directly on the device.

Supported displays include:

Controller	Resolution	SPI Speed
ST7789	240×240 / 320×240	40 MHz
ILI9341	240×320	64 MHz

Displays operate using 16-bit RGB565 color mode over SPI bus 0. The display driver abstraction allows multiple panels to be supported with consistent rendering behavior.

Camera Scanning

The camera is used solely for scanning QR codes. These QR codes represent structured requests from the online wallet, such as transaction signing requests or configuration payloads.

Camera input is treated as untrusted until decoded, validated, and presented to the user for approval. No automatic actions are taken based on scanned data without explicit user consent.

Optional MicroSD Usage

An optional MicroSD card may be used for non-sensitive device settings.

The MicroSD interface is explicitly **not** used for seed storage or private key persistence. Removal of the MicroSD card does not impact the security model of the device.

Power and Lifecycle

Stateless Operation

Cardano SeedSigner operates in a strictly stateless manner. All sensitive data, including private keys and derived signing material, exists only in volatile memory for the duration of the device's powered-on session.

The device does not persist keys, seeds, or signing state across reboots.

Memory Clearing on Power Loss

Loss of power results in the immediate destruction of all in-memory secrets. No recovery mechanism exists for restoring keys after shutdown, reboot, or unexpected power interruption.

This lifecycle model ensures that physical possession of the powered-off device does not grant access to sensitive material and reinforces the air-gapped security posture of the system.

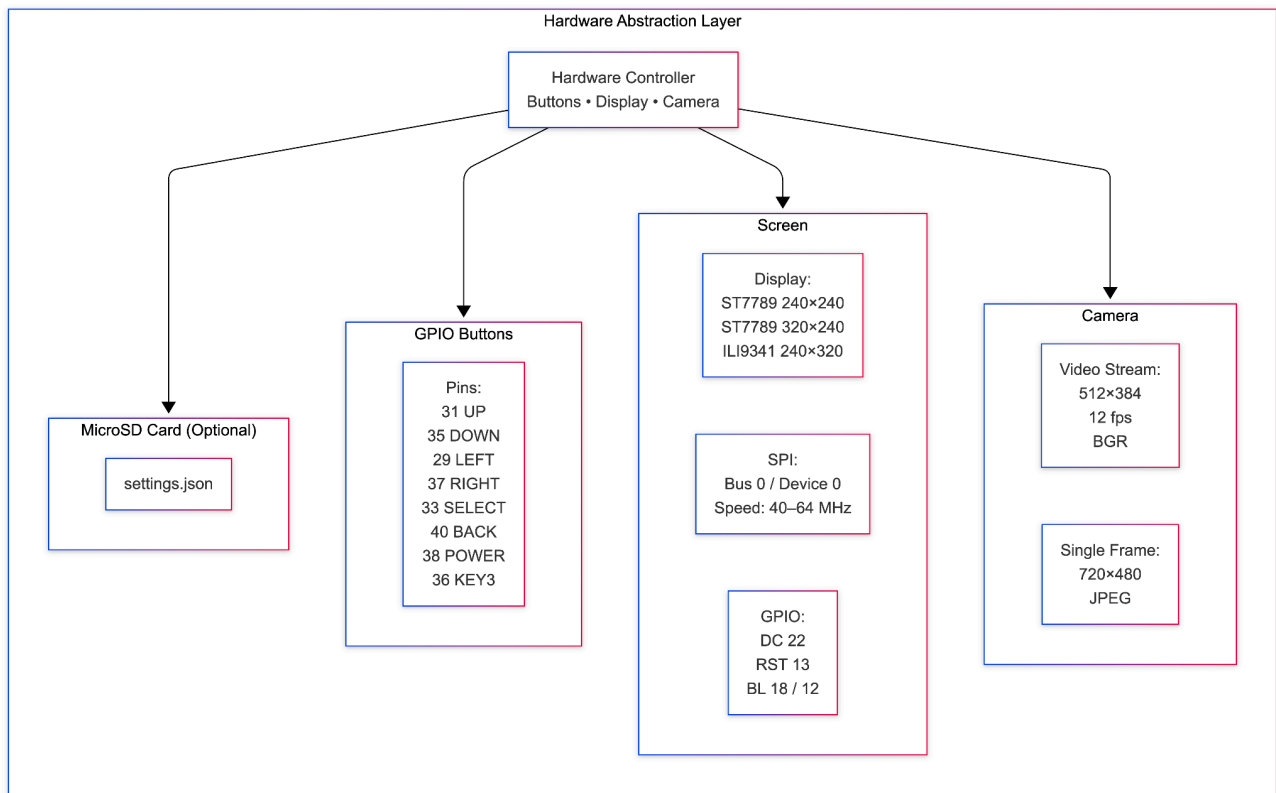


Figure 3 Hardware Abstraction Layer of Cardano SeedSigner.

Application Architecture

Application Structure

Cardano SeedSigner follows a simple, modular application architecture designed to minimize complexity and reduce the trusted computing base. The application is structured around a small number of well-defined components responsible for user interaction, state handling, and cryptographic operations.

At a high level, the system is organized into:

- **Views and screens**, which render information to the display and collect user input
- **Controllers**, which coordinate flows such as scanning, signing, and key management
- **Service components**, which perform cryptographic operations and QR encoding/decoding

User-facing functionality is expressed as explicit flows composed of multiple screens. Each flow is linear, interruptible, and requires direct user confirmation at all authorization boundaries. There is no background processing that can advance application state without user interaction.

This structure mirrors the original SeedSigner architecture, enabling reuse of proven design patterns while allowing Cardano-specific signing and data models to be introduced where required. Cardano CBOR parsing and cryptography are provided by the `cometa` library (Python bindings over `libcardano-c`).

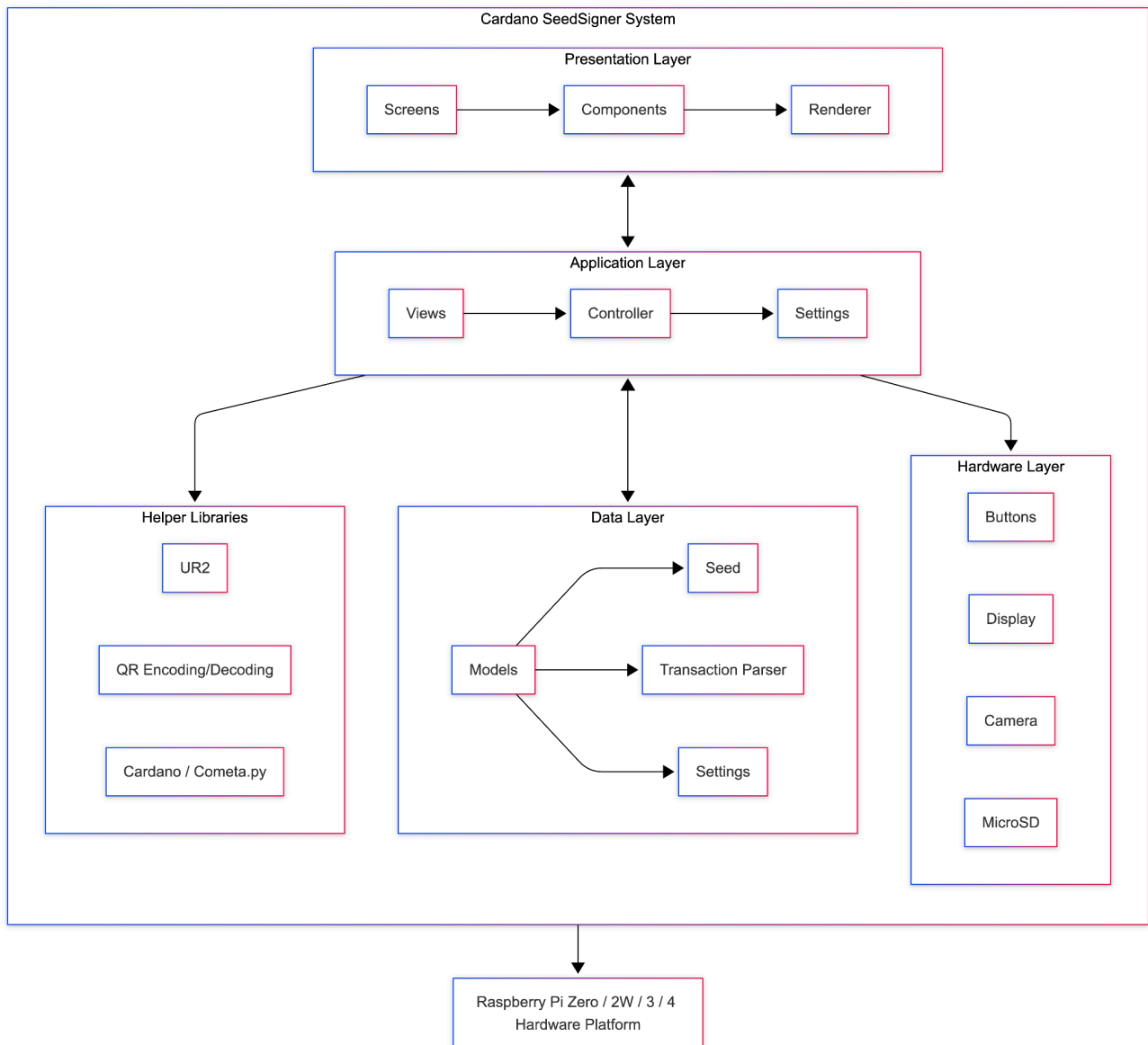


Figure 4 High-level application architecture of Cardano SeedSigner.

State Management

In-Memory Keys Only

All sensitive cryptographic material, including seeds, private keys, and derived signing keys, is held exclusively in volatile memory. Keys are loaded into memory only when explicitly provided by the user and remain available solely for the duration of the current session.

The application does not serialize, cache, or checkpoint private material to disk, removable media, or non-volatile storage.

No Persistent Secrets

Cardano SeedSigner does not persist secrets across application restarts or power cycles. When the device is shut down, restarted, or loses power, all in-memory state is destroyed.

Persistent storage, when present, is limited to non-sensitive configuration such as user interface preferences. The application enforces a strict separation between configuration state and cryptographic state to prevent accidental leakage or misuse of sensitive data.

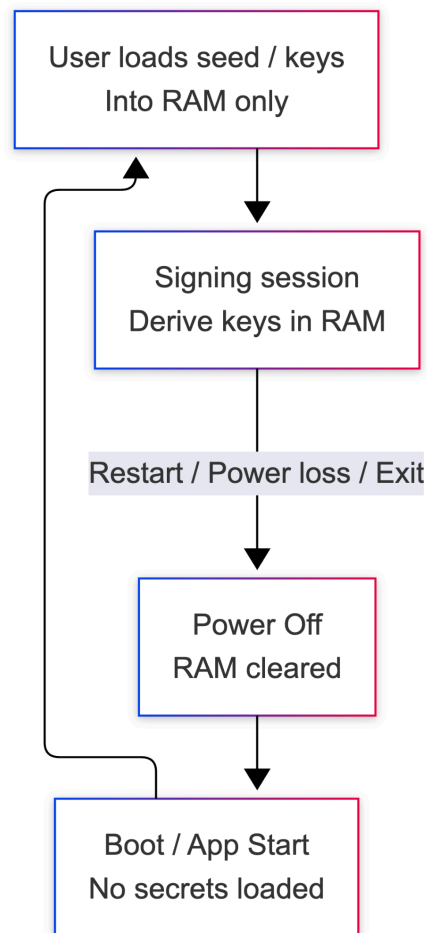


Figure 5 Seed / keys life cycle.

User Interface & Menus

User Interaction Principles

The Cardano SeedSigner user interface is intentionally minimal and security-driven. All interactions are performed locally on the device using physical controls and a trusted display.

Explicit User Consent

All security-relevant actions, such as loading a seed, signing a transaction, or exporting public information, require explicit user approval. The device never performs signing operations automatically or based on inferred intent.

There is no background processing, cached authorization, or unattended operation.

Human-Verifiable Information

Authorization decisions are based solely on information rendered on the device's screen. The interface prioritizes the display of verifiable data such as addresses, amounts, transaction hashes, and derivation paths. The device does not rely on labels, summaries, or metadata supplied by the online wallet unless they can be independently verified or clearly presented to the user.

Main Menu Overview

The main menu is the entry point for all device functionality and exposes only a limited, security-reviewed set of operations.

Menu Option	Description
Scan	Universal QR scanner for signing requests, account export requests, SeedQRs and settings
Seeds	Seed loading, in-session seed list, and per-seed operations
Tools	Utilities for offline seed generation
Settings	Device configuration and hardware settings
Power Off	Safe shutdown screen

The menu structure is deliberately flat to minimize navigation depth and reduce the likelihood of user error during critical operations.

Scan Flow

The Scan option provides a universal QR ingestion interface. All supported QR payloads are detected and routed based on content type.

QR Type	Resulting Action
Transaction signing request (cardano-tx-sig-req)	Transaction review and signing flow
Message signing request (cardano-cip8-sig-req)	CIP-8 message review and signing flow
Account export request (cardano-account-req)	Account public key derivation and export flow
SeedQR / CompactSeedQR	Seed loading flow
Settings (settings)	Settings import flow

Scanning a QR code never triggers an automatic action. All decoded information is presented for user review before any further processing occurs.

Seed Management

Seed material is managed explicitly and remains in volatile memory only for the duration of the session.

Operation	Description
Scan a SeedQR	Import a QR-encoded seed (SeedQR or CompactSeedQR)
Enter 12-word seed	Manual BIP-39 entry
Enter 15-word seed	Manual BIP-39 entry
Enter 24-word seed	Manual BIP-39 entry
Create a seed	Redirects to Tools menu (camera or dice entropy)

An optional BIP-39 passphrase can be attached during finalization. Once loaded, a seed can be used for signing operations during the active session. Seeds can be discarded at any time, immediately removing them from memory.

A loaded seed exposes the following operations:

Operation	Description
Scan transaction	Scan a transaction signing request for this seed
Sign message	Scan a CIP-8 message signing request for this seed
Export account key	Display the CIP-1852 account public key as an animated QR
Address explorer	Derive and inspect payment/stake addresses and DRep IDs
Backup seed	View seed words / export as SeedQR
Discard seed	Remove the seed from memory immediately

Tools Menu

The Tools menu provides offline utilities that do not require interaction with an online wallet.

Tool	Description
New Seed (Camera)	Generate entropy using camera image capture
New Seed (Dice)	Generate entropy using physical dice rolls
Calculate 12th / 24th Word	Compute the BIP-39 checksum word

These tools are designed to support fully offline key generation and verification workflows. The address explorer is reached from a loaded seed's menu, since it requires key material.

Transaction Signing Flow

The transaction signing flow is linear and review-driven.

1. A signing request is scanned via animated QR
2. The transaction body is parsed and validated; host ownership claims are cryptographically verified
3. An overview and a sequential, section-by-section review of every field present in the body is displayed
4. The user explicitly approves or rejects the request
5. The witness set is output as animated QR codes

The device does not assume ownership of inputs or correctness of balances beyond what is presented and cryptographically verifiable.

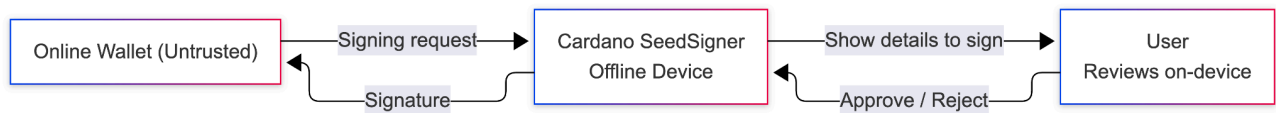


Figure 6 Transaction/Message sign flow.

Transaction Review Sections

The sequential review renders one page per item for every section present in the Conway-era transaction body. Nothing present in the body is omitted from review:

Section	Conway body key	Contents shown
Outputs	1	Address (Own/Foreign), ADA amount, native tokens, datum, reference script
Fee	2	Fee in ADA
TTL / Validity start	3 / 8	Validity interval bounds
Certificates	4	All Conway certificate types, with credential ownership badges
Withdrawals	5	Reward account and amount, with ownership badge
Auxiliary data hash	7	Hash preview
Mint / Burn	9	Per-policy mint and burn amounts with CIP-14 fingerprints
Script data hash	11	Hash preview
Collateral inputs	13	UTxO references
Required signers	14	Key hashes with ownership badges
Network id	15	Declared network
Collateral return	16	Address and amount, verified against the request's collateral return path
Total collateral	17	Amount
Reference inputs	18	UTxO references
Votes	19	Voter, governance action, vote, with DRep ownership badge
Proposals	20	All governance action types, including full protocol parameter change lists
Treasury / Donation	21 / 22	Amounts

Ownership Verification & Review Badges

Host claims about ownership are never trusted as labels; each is re-derived from the loaded seed and the result is shown as a badge:

- **Address (Own) + Verified Address:** an output listed in the request's `change_outputs` whose address, re-derived on-device from the claimed path (base addresses are verified against the account's standard stake key, role 2 index 0), matches the transaction body exactly. Anything else renders as **Address (Foreign)** with the full amount counted as leaving the wallet.
- **Verified Address** on the collateral return output, driven by the request's `collateral_return_path` (key 8).
- **Own Key / Unknown Key** under certificate credentials, required signers, and DRep voters, based on payment, stake, and DRep key hashes derived from the loaded seed.
- **Own Reward Account / Unknown Reward Account** on withdrawals.

Verification failures degrade safely: an unverifiable claim is displayed as foreign/unknown, inflating the apparent spend rather than hiding it.

Verified Assets

Native assets are identified by their CIP-14 fingerprint (`asset1. . .`). For a hand-curated list of well-known assets, the device additionally shows the ticker, the amount scaled by the issuer-attested number of decimals, and a green “Verified” badge. The list maps (`policy id`, `asset name`) pairs to display metadata, is network-scoped (a mainnet entry never labels a testnet asset and vice versa), and every entry is cross-checked against the Cardano Foundation token registry and an independent market listing of the policy id before inclusion.

The CIP-14 fingerprint remains visible for verified assets as an anti-spoofing measure. Assets not on the list show the raw integer amount with an explicit “Unknown decimals” warning.

Message Signing Flow

Message signing follows the same authorization model as transaction signing.

The user is shown:

1. An overview (requesting origin, credential type, payload preview)
2. The full signing credential: payment address, stake address, or DRep ID, classified by the derivation path role (role 3 renders a CIP-129 `drep1. . .` ID, role 2 a stake address, otherwise a payment address), after verifying the request's address is actually derived from the requested signing path
3. The full payload, rendered as text when printable, hex otherwise, with pagination

Signing proceeds only after explicit confirmation, and the resulting COSE signature is displayed as an animated QR code.

Settings & Device Configuration

The Settings menu exposes only non-sensitive configuration options.

General Settings

Setting	Description	Default
Language	Interface language	English
Persist settings	Store settings on the MicroSD card instead of in memory	Disabled
Camera rotation	Camera orientation (0/90/180/270)	180
Compact SeedQR	Enable CompactSeedQR backups	Enabled
Multi accounts	Browse and export keys for multiple CIP-1852 accounts (m/1852'/1815'/n'). When disabled, export requests for any non-zero account index are refused	Disabled
Multi credentials	Browse multiple stake keys and DRep keys per account (non-standard)	Disabled
MicroSD toast timer	Notification duration for card insert/remove	5 seconds
Show privacy warnings	Privacy caution screens (e.g. before key export)	Enabled
Show dire warnings	Dire caution screens (e.g. before seed display)	Enabled
QR density	Animated QR frame density	Medium
I/O Test	Button and display test	—
Donate	Project support information	—

Hardware Settings

Setting	Description
Display Type	ST7789 (240×240, default), ST7789 (320×240), ILI9341 (320×240, beta)
Invert Colors	Enabled / Disabled

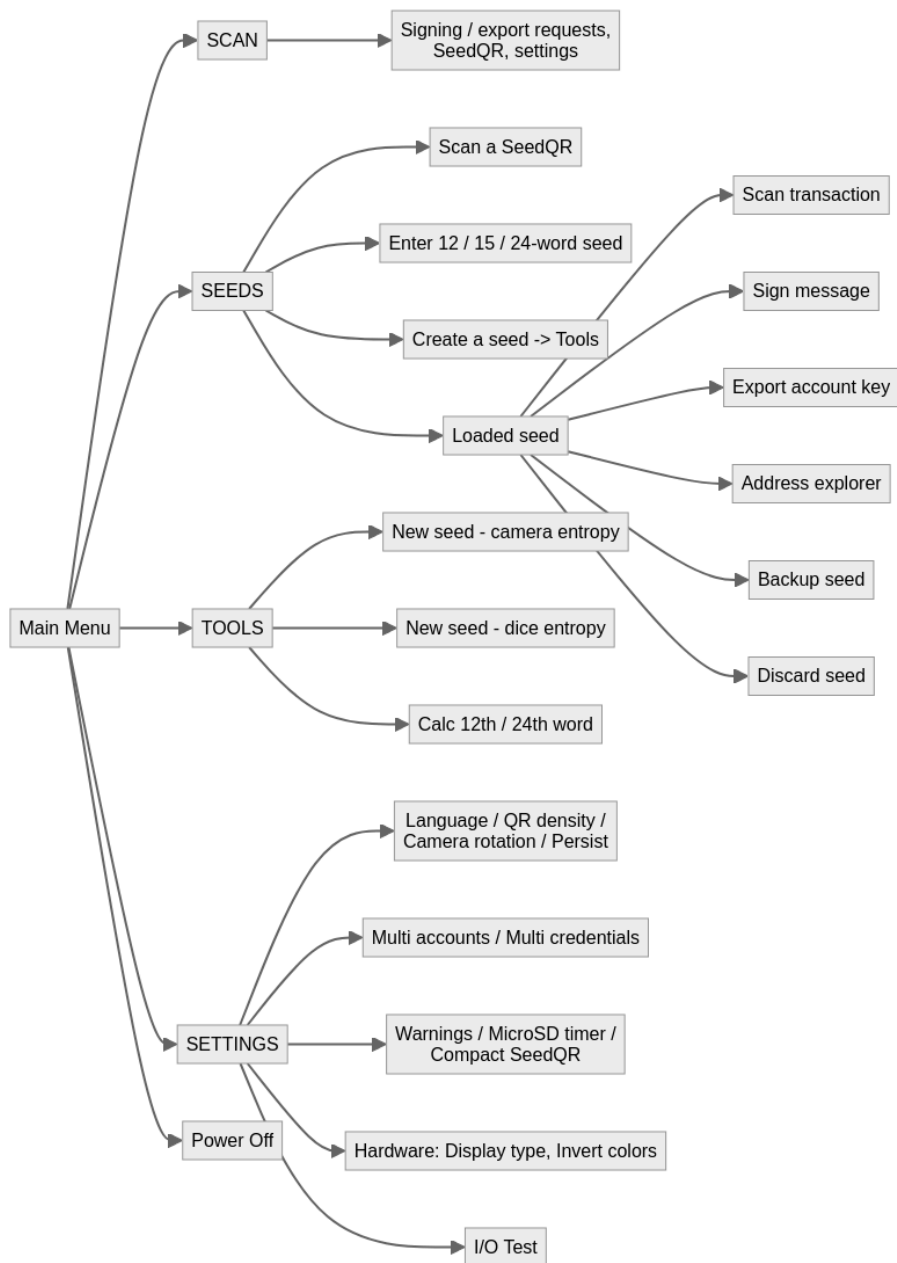


Figure 7 Menu tree.

Error and Warning Screens

The device includes dedicated error and warning screens to handle exceptional conditions such as invalid QR codes, unsupported requests, or hardware failures.

Errors are presented clearly and require user acknowledgment before returning to the main menu. No automatic retries or fallback actions are performed. Rejection screens name the exact reason a request cannot be signed; in particular, a transaction whose body declares a network id that contradicts the request's declared network is rejected before review.

Cardano Key & Signing Model

This section defines which Cardano keys the device supports, how they are derived and identified, and what it means for the user to authorize a signing operation on an air-gapped device.

Supported Key Types

Cardano SeedSigner operates exclusively on keys derived from the Ed25519 elliptic curve, using Cardano's hierarchical deterministic wallet scheme commonly referred to as Ed25519-BIP32. Unlike Bitcoin's secp256k1 curve, Ed25519 provides deterministic signatures (EdDSA) and is paired in Cardano with a hardened-only derivation model. This means that all private key derivations use hardened paths, and extended public keys cannot be used to derive child private keys or public keys beyond their intended scope.

Within this model, Cardano distinguishes between several functional key types derived from the same root seed. Payment keys are used to authorize spending of UTxOs. Stake keys are used for delegation, reward withdrawal, and governance-related actions where applicable. Additional specialized roles, such as governance representative (DRep) keys, are derived from the same hierarchical structure but serve distinct purposes in transaction authorization and on-chain participation. All signatures produced by the device are EdDSA signatures over Ed25519 and are valid only within the Cardano protocol context.

Derivation & Identification

Cardano wallets use BIP-39 mnemonics as the user-facing representation of entropy. The mnemonic, optionally combined with a passphrase, is converted into a binary seed using the standard BIP-39 key-stretching process. From this seed, Cardano SeedSigner derives keys using Cardano's registered derivation hierarchy, as defined by CIP-1852 and related specifications.

At a high level, Cardano derivation paths follow the structure:

```
m / 1852' / 1815' / account' / role / index
```

Where:

- 1852' identifies the Cardano HD wallet scheme,
- 1815' is Cardano's registered coin type,
- account' separates independent wallet accounts,
- role distinguishes payment (0), change (1), stake (2), and DRep (3) key roles,
- index selects a specific key within that role.

For example, a payment receiving key may be derived at:

```
m / 1852' / 1815' / 0' / 0 / 0
```

A stake key is derived at:

```
m / 1852' / 1815' / 0' / 2 / 0
```

SeedSigner uses these derivation paths only to derive keys required for the specific operation being authorized. Derivation paths carried in signing requests encode hardened components as the index value plus 0x80000000.

Fingerprints / Key Identification

Cardano SeedSigner computes a short, deterministic fingerprint for each loaded seed in order to provide a stable, human-verifiable identifier throughout the user session. The fingerprint is used exclusively for user orientation and request routing and does not participate in any cryptographic authorization or signing decisions.

The fingerprint is the standard 4-byte BIP-32 master key fingerprint: the first four bytes of HASH160 over the compressed secp256k1 master public key derived from the BIP-39 seed. This choice deliberately reuses the fingerprint every Bitcoin-family tool computes for the same mnemonic, giving one stable identity per seed across ecosystems. It is displayed as an 8-character hexadecimal identifier (e.g., A1B2C3D4).

The same 4 bytes are returned as `master_fingerprint` in the account export response, and hosts echo them back as the `xfp` fields of signing requests. This is how a request names which resident seed must sign: the device matches each `xfp` against the fingerprints of loaded seeds and refuses to sign with a seed the request does not name.

QR Communication & UR Encoding Model

Cardano SeedSigner uses animated QR codes exclusively, encoded using UR2 (Blockchain Commons Uniform Resources). UR is a transport format designed specifically for camera scanning: it wraps CBOR-encoded binary payloads into a QR-safe string using bytewords, and when the payload is large it uses fountain codes to split it into a stream of recoverable frames.

This section defines the request/response model, the UR frame format, the UR types used, the exact CBOR payload of every message, and the validation rules the device enforces.

Communication Pattern

All interactions follow a strict unidirectional flow at each step:

1. Host → Device: Request encoded as UR
2. User → Device: Visual inspection and explicit approval
3. Device → Host: Response encoded as UR

The device treats all incoming data as untrusted and relies on user review and cryptographic verification before producing any output.

UR URI Format

A UR is represented as:

```
ur:<type>[/<sequence>]/<body>
```

- `ur`: protocol prefix (lowercase)
- `<type>` payload type (e.g., `cardano-tx-sig-req`)
- `<sequence>` optional seqnum-seqlen for multipart animated URs
- `<body>` bytewords-encoded CBOR (minimal encoding)

Single-part example:

```
ur:cardano-tx-sig-res/hdeymejtswhhylkepmkhtsytsnoyoyaxaedsuttydmhhpktpmsrj  
tgwdpfnsboxgwlbawzuefywkdp1rsrjynbvgyabwjldapfcsdwkbrkch
```

Multipart example:

```
ur:cardano-tx-sig-req/2-5/lpaoaocsptcybgksksajkinjkcxinjkcxhscxjtjljyihjkjyc  
xjniyhjyhsjnieihjpsjzihkthsieiejzhsjniaishsiahsjy
```

Notes:

- seq_len is the count of unique fragments.
- seq_num can exceed seq_len once the encoder starts emitting redundant fountain frames.

Encoding Building Blocks

CBOR (RFC 8949) is used to represent structured data compactly.

Bytewords converts bytes into text suitable for QR scanning. UR uses minimal bytewords, encoding each byte into 2 characters (first+last letters of a 4-letter word), plus a CRC checksum.

Fountain codes (LT codes) enable robust animated scanning:

- The sender can emit unlimited frames,
- The receiver can miss frames,
- And still recover the original message after collecting roughly $\sim 1.75\times$ the unique fragment count in typical conditions.

UR Types Used by Cardano SeedSigner

Cardano SeedSigner reuses existing UR types where applicable (seed), and introduces Cardano-specific request/response types. **The UR type string is the payload discriminator. Message bodies are plain (untagged) CBOR maps**; the registry ids below are reserved numbers for documentation and future registration and are never emitted on the wire. This follows the Blockchain Commons convention for typed URs.

Purpose	UR Type	Registry id	Direction
Seed import	seed	40300	Incoming
Transaction signing request	cardano-tx-sig-req	88000	Incoming
CIP-8 message signing request	cardano-cip8-sig-req	88001	Incoming
Transaction signing response	cardano-tx-sig-res	88002	Outgoing
CIP-8 message signing response	cardano-cip8-sig-res	88003	Outgoing
Account public key export request	cardano-account-req	88004	Incoming
Account public key export	cardano-account	88005	Outgoing
Settings payloads	settings	88006	Both

Payload Definitions (CDDL)

Account public key export request (cardano-account-req)

```
CardanoAccountRequest = {  
  1: tstr,           ; request_id (UUID, echoed in the response)  
  ? 2: tstr,         ; origin (wallet name, e.g. "Eternl")  
  3: [* uint],       ; account_indices (one or more, each below 2^31)  
  ? 4: uint          ; key_purpose (default 1852 = CIP-1852)  
}
```

When the Multi accounts setting is disabled (the default), any request naming an account other than index 0, or more than one account, is refused on-device.

Account public key export (cardano-account)

```
CardanoAccountResponse = {  
  1: tstr,           ; request_id (echoed; "" for menu-initiated export)  
  2: bstr,           ; master_fingerprint (4 bytes, BIP-32)  
  3: [* AccountKey], ; one entry per exported account  
  ? 4: tstr          ; device_label  
}  
  
AccountKey = {  
  1: uint,           ; account index  
  2: bstr,           ; account xpub (64 bytes: 32 pubkey || 32 chaincode)  
  3: [* uint]        ; derivation path, e.g. [1852', 1815', account']  
}
```

The response shape mirrors Blockchain Commons crypto-multi-accounts: the master fingerprint identifies the seed, and each account entry carries the CIP-1852 account-level extended public key from which the host derives payment, stake, and DRep keys (soft derivation only).

Transaction signing request (cardano-tx-sig-req)

```

CardanoTxSignRequest = {
  1: tstr,                ; request_id (UUID)
  ? 2: tstr,              ; origin (optional wallet name)
  3: bstr,                ; sign_data (transaction body CBOR)
  4: [* SigningInput],    ; inputs
  5: [* ChangeOutput],    ; change_outputs
  6: [* ExtraSigner],     ; extra_signers
  7: uint,                ; network (0 = testnet, 1 = mainnet)
  ? 8: SignerPath         ; collateral_return_path
}

SigningInput = {
  1: bstr .size 32,       ; tx_hash of the spent UTxO
  2: uint,                ; output index within that transaction
  ? 3: bstr .size 4,      ; xfp (master fingerprint of the owning seed)
  ? 4: [* uint]           ; derivation path of the spending key
}

ChangeOutput = {
  1: uint,                ; output index in the transaction body
  2: [* uint],            ; derivation path used to verify the address
  ? 3: bstr .size 4       ; xfp of the seed that owns this change
}

ExtraSigner = SignerPath = {
  1: bstr .size 4,        ; xfp
  2: [* uint]             ; derivation path
}

```

sign_data is the raw Conway-era transaction body CBOR; the device parses it directly and signs its Blake2b-256 hash. A script-locked input (multisig / Plutus) carries no xfp/path; its witnesses are requested via extra_signers (e.g. minting policy keys, required signers). The request deliberately carries **no resolved inputs**: the device cannot verify host-supplied UTxO amounts against the chain, so unverifiable data is not presented as fact; the review shows outputs, fee, and every other body field instead.

Transaction signing response (cardano-tx-sig-res)

```

CardanoTxSignResponse = {
  1: tstr,                ; request_id (echoed)
  2: bstr                 ; vkey_witness_set_cbor
}

```

The response carries a standard Cardano vkey witness set (one Ed25519 signature over the transaction body hash per required key, deduplicated). The host merges it into the full transaction

for submission.

CIP-8 message signing request (cardano-cip8-sig-req)

```
CardanoCip8SignRequest = {
  1: tstr,                ; request_id (UUID)
  ? 2: tstr,              ; origin (optional wallet name)
  3: bstr,                ; message_payload
  4: bstr,                ; xfp (4 bytes; empty = unspecified)
  5: SigningPath,         ; required_signing_path
  ? 6: bstr               ; address_bytes (sign-with address or key hash)
}

SigningPath = {
  1: uint,                ; index
  2: [* uint]            ; derivation path
}
```

address_bytes is the credential the signature is bound to: a full payment or stake address, or a bare 28-byte key hash for DRep signing. The device refuses to sign when the key derived from required_signing_path does not actually control address_bytes, preventing a host from showing the user one credential and binding the signature to another.

CIP-8 message signing response (cardano-cip8-sig-res)

```
CardanoCip8SignResponse = {
  1: tstr,                ; request_id (echoed)
  2: bstr,                ; cose_sign1_cbor
  3: bstr                 ; cose_key_cbor
}
```

Settings (settings)

```
ur:settings/lpadaocsptcybgkskgsajk...
```

The settings payload is the device's non-sensitive configuration map (language, QR density, camera rotation, display type, etc.). Cryptographic material is never part of a settings payload.

CIP-8 COSE Wire Format

The CIP-8 implementation emits the exact wire format used by Ledger (app-cardano) and Keystone, which is also verified by Lace. This maximizes interoperability across the hardware wallet ecosystem:

```

COSE_Sign1 = [
  protected:  bstr .cbor {
    1: -8,                ; alg: EdDSA
    "address": bstr        ; the bound credential (see below)
  },
  unprotected: { "hashed": false },
  payload:      bstr,      ; the message
  signature:    bstr       ; Ed25519 over Sig_structure
]

Sig_structure = [
  "Signature1",
  protected,          ; same bstr as above
  external_aad: bstr .size 0,
  payload
]

COSE_Key = {
  1: 1,                ; kty: OKP
  2: bstr,              ; kid: same value as "address"
  3: -8,                ; alg: EdDSA
  -1: 6,                ; crv: Ed25519
  -2: bstr              ; x: Ed25519 public key (32 bytes)
}

```

The protected header contains exactly two fields (no kid), and the "address" value is the full address bytes for payment/stake credentials or the bare 28-byte key hash for DRep credentials. The device classifies the credential for display by the derivation path role, so a DRep key wrapped in an enterprise-style address is still presented as a DRep ID (CIP-129 drep1. . .).

Validation Rules

Every incoming payload is validated before anything is shown to the user; a malformed request is rejected with a specific error:

- xfp fields must be exactly 4 bytes (or absent/empty where the schema allows an unspecified owner). An input that declares a derivation path must also declare the owning xfp.
- tx_hash must be exactly 32 bytes.
- Derivation paths carry 1 to 10 components (CIP-1852 paths have 5; the cap rejects abusive inputs); each component fits in 32 bits; hardened components are encoded as value + 0x80000000.
- account_indices must be non-empty and each below 2^{31} ; the only supported key_purpose is 1852.

- The transaction body must parse as a Conway-era body; a body-declared network id (key 15) that contradicts the request's network field is rejected before review.
- Signing requests are matched to a loaded seed by xfp; a request naming no loaded seed cannot be signed.

Fragmentation & Animated QR

UR payloads exceeding a single QR frame are automatically fragmented using fountain codes. This provides:

- Loss tolerance: Frames may be skipped or scanned out of order
- Progressive decoding: Completion without all frames
- User control: Scanning can be interrupted safely at any time

Each UR frame includes:

- A sequence identifier
- Payload fragment
- CRC checksum for integrity

```
ur:cardano-tx-sig-req/3-7/lpaxaycsptcybgksk...
```

Compatibility & Extensibility

Cardano SeedSigner follows established UR conventions to ensure interoperability. Where Cardano-specific payloads are required (e.g., transaction and message signing), new UR types are defined without modifying the underlying UR transport or encoding rules.

Future extensions may introduce additional Cardano UR types while preserving backward compatibility and the same animated QR communication model.

User & System Interaction Diagrams

Seed Import

The user initiates the flow directly on the device by scanning a SeedQR or manually entering mnemonic words. The seed is validated locally, loaded into volatile memory, and made available for signing operations during the current session only. No external actor is involved, and the seed material is never transmitted back to the hot wallet.

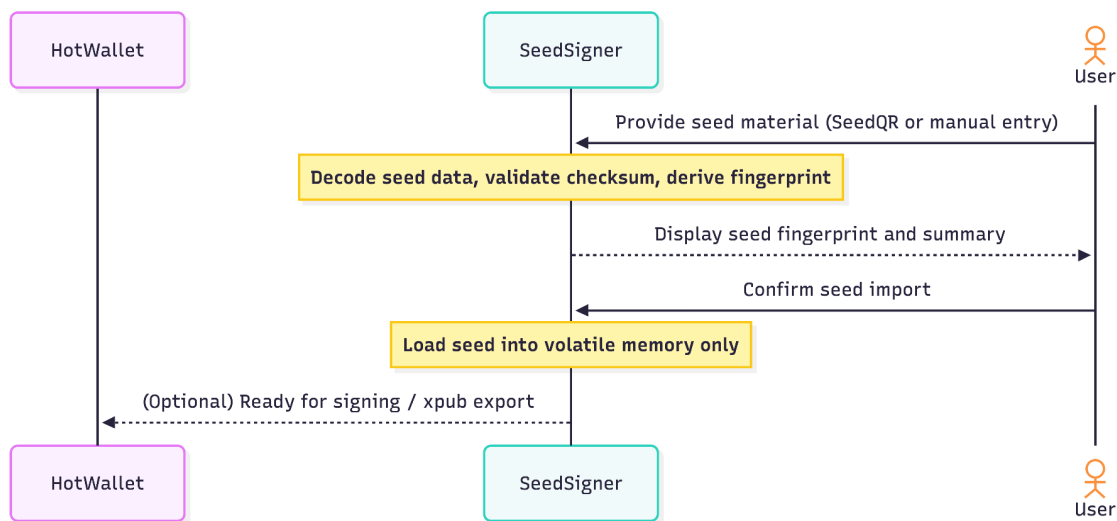


Figure 8 Seed import flow.

Extended Public Key (xpub) Export

After the user selects an export option, SeedSigner derives the requested extended public key according to the specified Cardano derivation path. The derived public data is encoded as a QR payload and displayed to the user. The hot wallet scans the QR to import the xpub for watch-only or account setup purposes. Private key material never leaves the device.

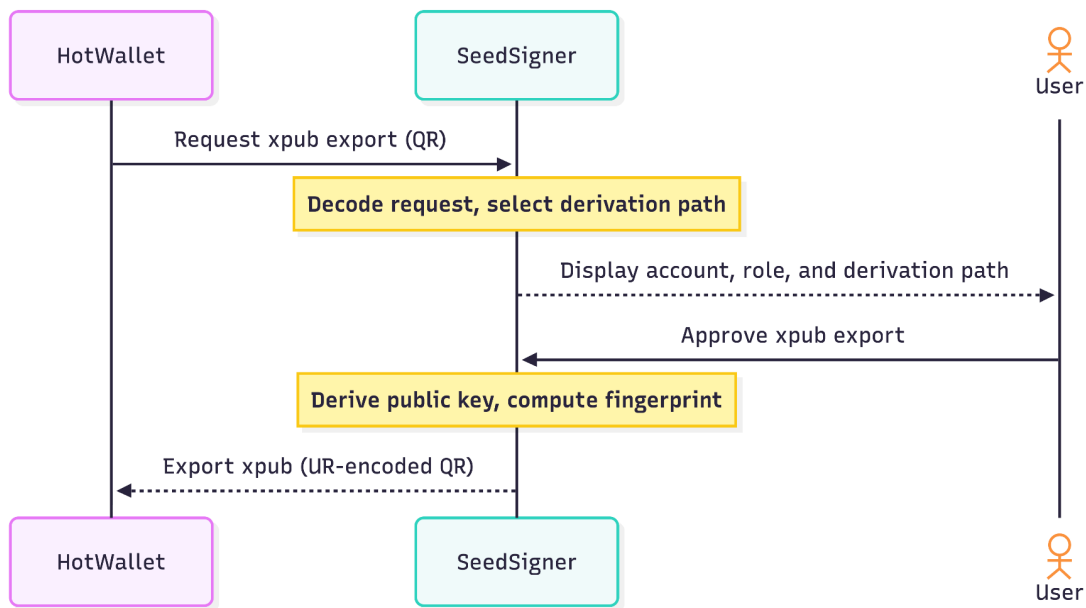


Figure 9 Extended public key export flow.

Settings Import/Export

The user initiates the export from the settings menu, after which SeedSigner serializes only configuration state such as language, QR density, or hardware preferences. The exported settings are presented as a QR code that can be scanned by a hot wallet or companion application. Cryptographic material is explicitly excluded from this process.

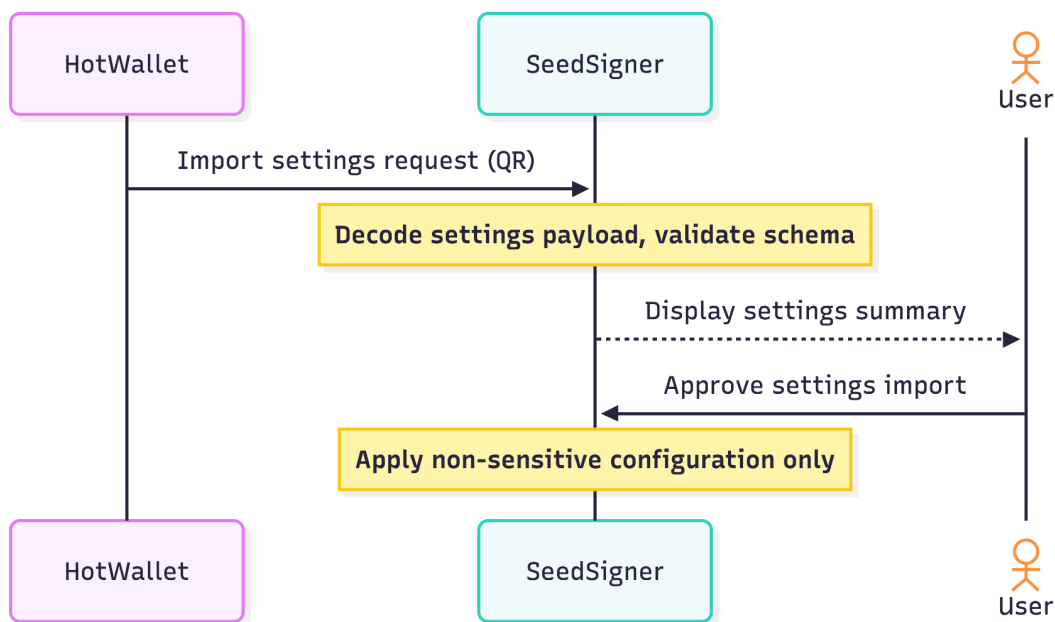


Figure 10 Settings import flow.

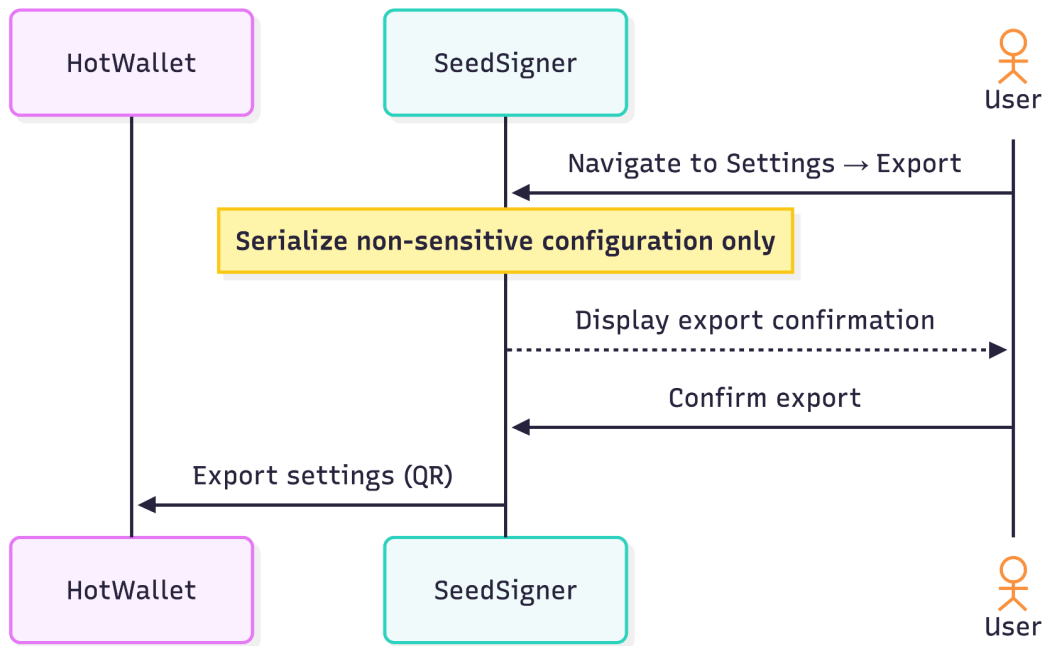


Figure 11 Settings export flow.

Transaction Signing

The hot wallet submits a transaction signing request to SeedSigner using a QR-encoded payload. SeedSigner decodes and validates the request, verifies every ownership claim it can, then presents a human-verifiable review of the transaction to the user. Signing proceeds only after explicit user approval, and the resulting witness set is returned to the hot wallet via QR code.

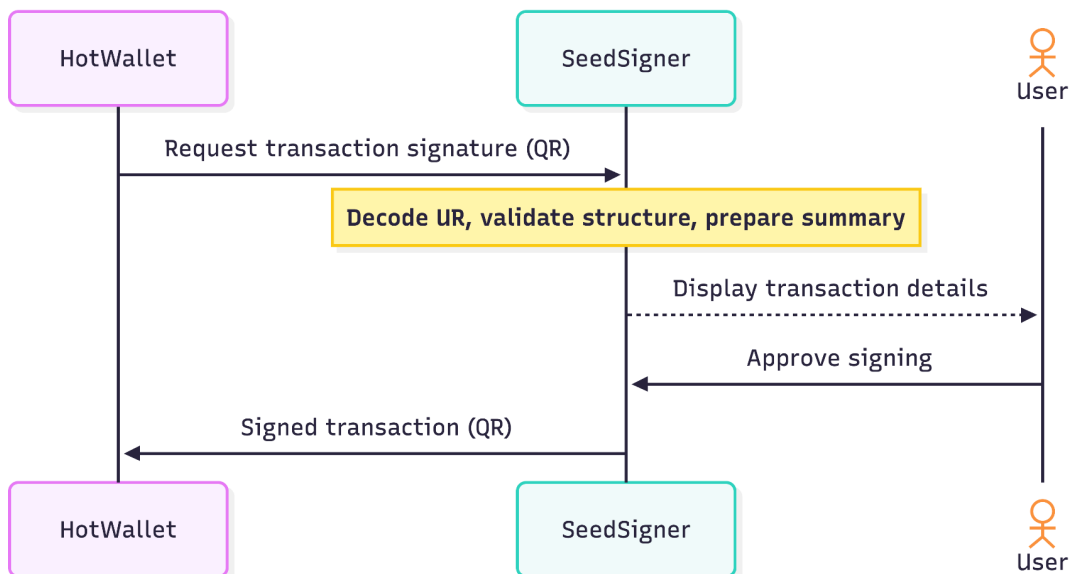


Figure 12 Transaction signing flow.

CIP-8 Message Signing

A hot wallet submits a message signing request to SeedSigner using a QR-encoded payload that specifies the message and signing context. SeedSigner decodes the request, displays the full message content and the bound credential to the user, and waits for explicit approval. Upon confirmation, SeedSigner derives the appropriate key and produces an Ed25519 signature compliant with CIP-8 in the Ledger/Keystone wire format, returning the COSE_Sign1 and COSE_Key to the hot wallet via QR code.

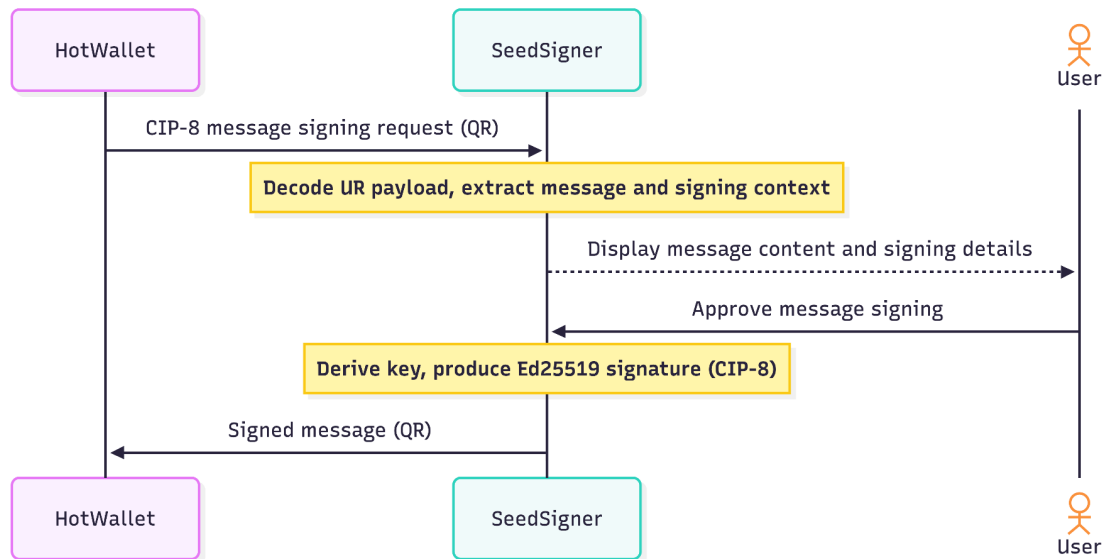


Figure 13 CIP-8 Message signing flow.